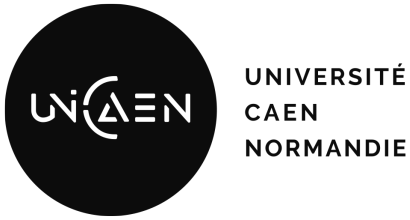
Suivi des connaissances - Projet Multi-Agents

Tom David, Pasquier Titouan, Guillais Marius, Chepy Maxim

Semestre 2 - 2025/2026



Sommaire

1 Introduction **3**

2 Présentation des scénarios **4**
2.1 Le jeu de la ville 4
2.1.1 Règles 4
2.1.2 Situation intéressante : le scénario mixte 4
2.2 Hanabi 7
2.2.1 Règles 7
2.2.2 Situation intéressante : un indice bien placé 7
2.2.3 Tours clés illustrés (seed 4) 8
2.2.4 Analyse des performances sur 20 000 parties 9

3 Abstraction générale **10**
3.1 Les concepts fondamentaux 10
3.2 État de croyance (Belief State) 10

4 Réalisation technique **12**
4.1 Le framework générique (`core`) 12
4.1.1 Structure globale 12
4.1.2 Belief State et filtrage 12
4.1.3 Organisation des fichiers 13
4.2 Jeu de la ville 13
4.2.1 Modélisation 13
4.2.2 Politique PCC et raisonnement du premier ordre 14
4.2.3 Raisonnement du second ordre 14
4.3 Hanabi 15
4.3.1 Modélisation 15
4.3.2 Domaines de croyance 15
4.3.3 Politique et raisonnement de second ordre 16

5 Perspectives **17**

6 Conclusion **17**
6.1 Notions apprises 18
6.2 Répartition des tâches 18

1 Introduction

Un **système multi-agents** est un ensemble d'agents autonomes qui agissent dans un environnement partagé et interagissent selon des règles définies. L'idée centrale est de décentraliser la résolution d'un problème complexe : plutôt qu'un seul programme centralisé qui décide de tout, chaque agent prend ses propres décisions en fonction de ce qu'il observe localement.

Ce type de modélisation s'inspire de comportements naturels comme les fourmis ou les abeilles, et se retrouve dans de nombreux domaines : robotique coopérative, jeux vidéo, simulation de foules, logistique, etc.

Ce projet s'intéresse à un cas particulièrement contraignant : des agents qui **ne peuvent pas communiquer directement** entre eux. Chaque agent ne dispose que de ses propres observations pour décider de ses actions. Pourtant, une coordination implicite reste possible si les agents raisonnent sur ce que les autres savent et sur la façon dont ils vont probablement se comporter.

L'objectif est de concevoir un **framework générique** en Java, capable de simuler n'importe quel scénario respectant ce modèle, puis de l'illustrer sur deux scénarios concrets : le **jeu de la ville** (problème du rendez-vous) et **Hanabi** (jeu coopératif à information asymétrique).

2 Présentation des scénarios

Avant de présenter l'abstraction générale, nous décrivons les deux jeux qui servent de terrain d'expérimentation. L'objectif est de mettre en évidence les situations intéressantes qui motivent les choix de conception.

2.1 Le jeu de la ville

2.1.1 Règles

Plusieurs agents se trouvent dans une ville modélisée par un graphe : les nœuds sont des lieux (intersection, métro, pont, etc.) et les arêtes sont des chemins praticables. Tous les agents doivent se retrouver en un **lieu de rendez-vous** commun.

À chaque tour, un agent peut :
— se **déplacer** vers un lieu voisin,
— **rester** sur place,
— **déposer un message** sur sa case actuelle, qui pourra être lu par un agent passant par ce lieu plus tard.

La difficulté vient du fait que certains lieux peuvent être **fermés** : si un agent tente de se déplacer vers un lieu fermé, il reste sur place. Il apprend ainsi qu'il y a un problème, mais les autres agents ne le savent pas encore. Le seul moyen de partager cette information est de déposer un message sur un lieu stratégique.

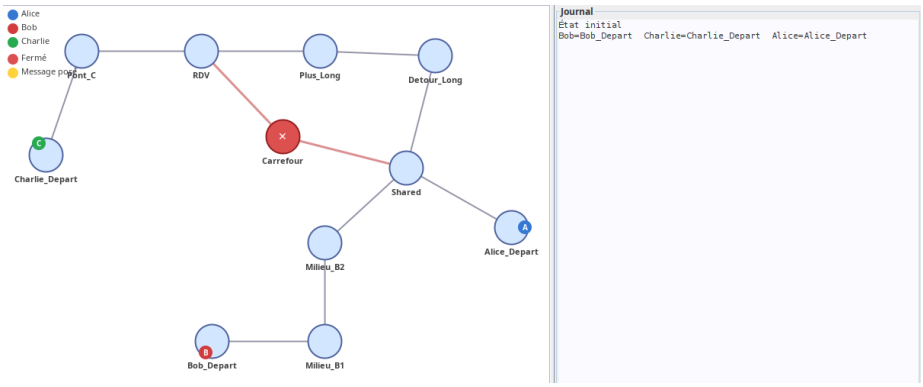
2.1.2 Situation intéressante : le scénario mixte

Le scénario par défaut met en jeu trois agents : Alice, Bob et Charlie.

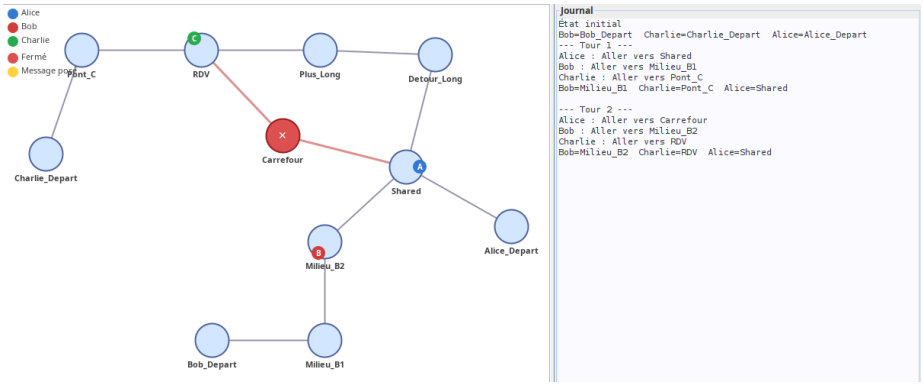
Au départ, chaque agent croit que tous les chemins sont ouverts et calcule le plus court chemin vers le RDV. Alice part vers Carrefour (chemin le plus court), tente de s'y rendre et échoue : elle découvre que Carrefour est fermé. Bob arrive à Shared plus tard depuis un chemin plus long ; Alice décide de lui laisser un message sur Shared pour l'avertir, ce qui lui économise un tour.

En revanche, Charlie emprunte un chemin complètement indépendant : Alice ne lui laisse pas de message car il n'en a pas besoin. C'est le **raisonnement du second ordre** : Alice raisonne sur ce que chaque autre agent sait déjà avant de décider si un message est utile.

Les captures suivantes illustrent ce déroulement tour par tour.

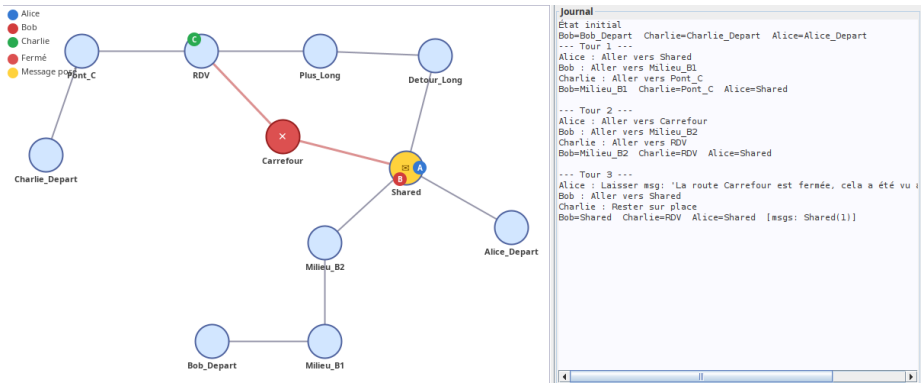


(a) État initial : Alice à Alice_Depart, Bob à Bob_Depart, Charlie à Charlie_Depart. Carrefour est fermé (en rouge).

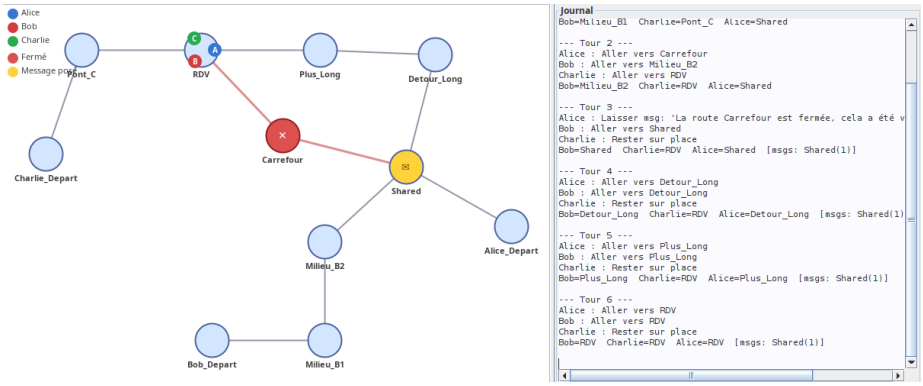


(b) Tour 2 : Alice tente Carrefour et échoue (reste à Shared). Elle découvre que le lieu est fermé. Bob progresse vers Milieu_B2, Charlie atteint RDV.

FIGURE 1 – Scénario mixte (1/2) : état initial et découverte de la fermeture.



(a) Tour 3 : Alice laisse un message sur Shared (icône jaune). Bob arrive à Shared et le lit : il sait que Carrefour est fermé sans l'avoir tenté lui-même.



(b) Tour 6 : Alice et Bob rejoignent Charlie au RDV en empruntant le détour. La simulation se termine.

FIGURE 2 – Scénario mixte (2/2) : dépôt du message et arrivée au rendez-vous.

2.2 Hanabi

2.2.1 Règles

Hanabi est un jeu de société coopératif. L'objectif est de constituer, pour chacune des 5 couleurs, une pile de cartes ordonnées de 1 à 5, soit un score maximum de 25 points.

La particularité : chaque joueur **voit les cartes des autres mais pas les siennes**. À chaque tour, le joueur actif peut :

- **poser** une de ses cartes sur une des piles (cependant si la carte n'est pas de la bonne couleur et/ou mauvais numéro alors on rajoute une erreur),
- **défausser** une carte pour récupérer un jeton d'indice,
- **donner un indice** à un autre joueur : indiquer toutes les cartes d'une certaine couleur, ou toutes les cartes d'une certaine valeur dans sa main.

Le jeu s'arrête à 3 erreurs (défaite immédiate) ou lorsque la pioche est épuisée. Il est également important de noter que chaque joueur possède 5 cartes dans sa main et que si il en défausse/joue une alors il récupère la carte du dessus de la pioche si elle n'est pas vide.

2.2.2 Situation intéressante : un indice bien placé

Voici un exemple typique de raisonnement en action. Supposons que l'agent A observe que l'agent B a un 1 rouge en main, et que la pile rouge est vide : ce 1 rouge est immédiatement jouable.

Avant de donner un indice, A se pose la question : *vaut-il mieux indiquer la couleur rouge ou la valeur 1 ?* Si B a déjà reçu un indice de valeur 1 lors d'un tour précédent, ses croyances sur ses cartes de valeur 1 sont déjà réduites ; un deuxième indice valeur serait peu informatif. En revanche, un indice couleur rouge lui apprendrait que sa carte est rouge, ce qui combiné à ce qu'il sait déjà (valeur 1) lui permet de déduire qu'elle est jouable immédiatement.

C'est ce raisonnement que nous appelons **raisonnement de second ordre** côté Hanabi : l'agent A modélise ce que B sait déjà pour choisir l'indice le plus informatif, plutôt que de choisir aléatoirement.

2.2.3 Tours clés illustrés (seed 4)

La figure 3 illustre ce mécanisme sur deux tours consécutifs d'une partie réelle (graine 4).

Tour 1 - Bob donne un indice. Bob observe la main d'Alice : elle contient quatre cartes de valeur 1 (jaune, rouge, vert, bleu) et une carte de valeur 3. Toutes les piles sont vides, donc tous les 1 sont immédiatement jouables. Bob raisonne en second ordre : indiquer « valeur 1 » touche quatre emplacements en une seule action, ce qui est plus informatif qu'un indice couleur qui n'en toucherait qu'un ou deux. Il choisit donc « indique valeur des cartes 1 ».

Tour 2 - Alice réduit ses domaines et joue. Après réception de l'indice, les domaines d'Alice se transforment fortement :

— Domaines 0, 1, 3, 4 : réduits à **uniquement des 1** (toutes couleurs, 15 cartes chacun), ces emplacements contiennent bien des valeur 1.

— Domaine 2 : ne contient **aucun 1** (que des 2, 3, 4, 5) cet emplacement n'est pas un 1.

Toutes les piles étant vides, chaque 1 est jouable. Alice pose la carte en position 0 avec certitude. Le score passe de 0 à 1 (jaune 1 posé), sans aucune erreur.



FIGURE 3 – Tours 1 et 2 (seed 4) : Bob donne l'indice « valeur 1 » après avoir modélisé les domaines d'Alice (second ordre) ; Alice réduit ses domaines et pose une carte en toute certitude.

2.2.4 Analyse des performances sur 20 000 parties

Sur 20 000 parties jouées avec des graines aléatoires, la politique obtient une **moyenne de 18,55/25**, ce qui représente environ 74% du score maximum. Le meilleur score obtenu est **25/25** (partie parfaite) et le pire est **5/25**.

Meilleure partie (seed `7950951865960141691`, score 25/25). Cette partie illustre le fonctionnement idéal de la politique. Les agents alternent indices et poses de cartes tout au long des 86 tours, sans jamais commettre d'erreur. Les domaines de croyance se réduisent progressivement et de façon ciblée, permettant à chaque agent de poser ses cartes avec certitude. La pioche n'est pas entièrement épuisée (6 cartes restantes), signe que la coordination a été suffisamment efficace pour compléter les cinq piles avant la fin.

Ce cas favorable s'explique par une distribution initiale propice : les mains contiennent des cartes de valeur faible immédiatement jouables dès les premiers tours, ce qui permet de démarrer toutes les piles rapidement et d'enchaîner les poses sans blocage.

Pire partie (seed `-7009734669824303062`, score 5/25). Le jeu de Hanabi comporte une part d'aléatoire inhérente à l'ordre de la pioche. Dans cette partie, la distribution initiale est particulièrement défavorable : les mains des deux agents contiennent principalement des cartes de valeur haute (4 et 5) alors que les piles sont toutes vides. Sans cartes de valeur 1 ou 2 disponibles, les agents ne peuvent pas démarrer la majorité des piles et se retrouvent à enchaîner les défausses pour renouveler leurs mains. La pioche s'épuise avant que la situation se débloque : deux couleurs restent à 0 carte posée (blanc et jaune), et le score final de 5 est atteint sans aucune erreur, uniquement par manque de cartes jouables au bon moment. Ce type de cas est rare mais illustre la limite fondamentale de toute politique face à une distribution initiale défavorable.

3 Abstraction générale

Les deux scénarios, bien que très différents, partagent une structure commune. Nous formalisons ici les concepts clés du framework.

3.1 Les concepts fondamentaux

État. L'état du jeu représente la situation complète à un instant donné : positions des agents, contenu des mains, routes fermées, etc. On note l'ensemble des états possibles $\mathcal{S}$.

Action. À chaque tour, chaque agent choisit une action parmi un ensemble d'actions disponibles $\mathcal{A}$, qui dépend de l'état courant. L'ensemble des choix simultanés de tous les agents forme une **action jointe**.

Transition. Le jeu fait évoluer l'état en fonction de l'action jointe. On note cette fonction de transition :

$$T : \mathcal{S} \times \mathcal{A}^n \to \mathcal{S}$$

où n est le nombre d'agents.

Observation. Après chaque transition, chaque agent reçoit une **observation** partielle de l'état résultant. Cette observation peut être différente pour chaque agent : dans Hanabi, un agent voit les mains des autres mais pas la sienne ; dans le jeu de la ville, un agent sait s'il a réussi à se déplacer ou non. On note l'ensemble des observations possibles $\mathcal{O}$.

Historique. L'historique d'un agent est la séquence de toutes ses paires (action, observation) depuis le début de la partie. C'est la seule source d'information dont dispose un agent sur le déroulement du jeu :

$$h_t = \big[(a_1, o_1),\, (a_2, o_2),\, \ldots,\, (a_t, o_t)\big]$$

Politique. La politique est la fonction de décision de l'agent. Elle prend en entrée l'historique et la liste des actions disponibles, et retourne l'action à jouer :

$$\pi : \mathcal{H} \times 2^{\mathcal{A}} \to \mathcal{A}$$

Chaque scénario fournit sa propre implémentation de politique, adaptée au problème.

3.2 État de croyance (Belief State)

Dans les deux scénarios, l'agent ne peut pas observer l'état global directement. Il doit donc raisonner sur l'ensemble des états qui seraient **compatibles** avec

tout ce qu'il a observé jusqu'ici. C'est ce qu'on appelle l'**état de croyance** (ou *belief state*).

Formellement, à chaque instant t, l'état de croyance B_t est un ensemble d'états :

$$B_t \subseteq \mathcal{S}$$

tel que tout état $s \in B_t$ aurait pu produire exactement les mêmes observations que celles reçues par l'agent depuis le début de la partie.

La mise à jour de B_t en B_{t+1} suit le principe de filtrage bayésien : on garde uniquement les états successeurs dont l'observation simulée correspond à l'observation réelle reçue par l'agent. Ce mécanisme est générique et s'applique à n'importe quel scénario.

4 Réalisation technique

4.1 Le framework générique (`core`)

4.1.1 Structure globale

Le package `core` contient l'ensemble des abstractions qui constituent le framework. Chaque scénario l'étend sans modifier le cœur.

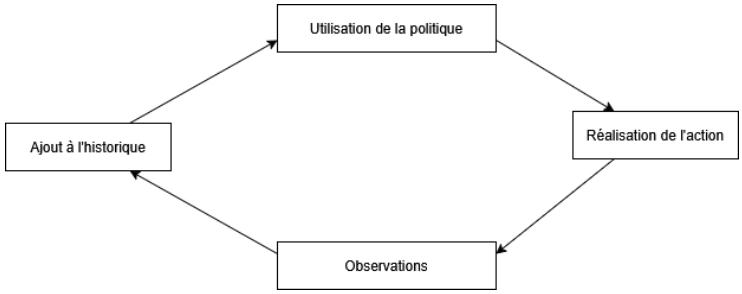


FIGURE 4 – Cycle de simulation : à chaque pas de temps, chaque agent choisit une action via sa politique, le jeu calcule le nouvel état, et chaque agent reçoit une observation.

Le **simulateur** orchestre la boucle de jeu :

1. chaque agent consulte sa politique pour choisir une action,
2. le jeu calcule le nouvel état à partir de l'action jointe,
3. chaque agent reçoit une observation et la stocke dans son historique.

La classe abstraite `Jeu` impose aux scénarios de définir : l'état initial, la condition de fin, la fonction de transition, les actions disponibles par agent, et la génération des observations. Cela garantit que n'importe quel scénario peut tourner dans le même simulateur sans modification.

4.1.2 Belief State et filtrage

Le `BeliefState<E>` est une classe générique paramétrée par le type d'état `E`. Il maintient la liste des états compatibles et se met à jour à chaque pas de temps via un `BeliefStateFilter<E>`.

Le filtre fonctionne ainsi :

1. Pour chaque état compatible, il génère toutes les transitions possibles : l'action de l'agent observateur est fixée (on la connaît puisqu'on l'a choisie), et on énumère toutes les combinaisons d'actions des autres agents.
2. Pour chaque transition, il simule l'observation que l'agent aurait reçue via `simulerObservation`.
3. Il ne conserve que les états successeurs dont l'observation simulée correspond à l'observation réelle.

L'intérêt de cette généricité est qu'un scénario bénéficie automatiquement du mécanisme de belief state dès qu'il implémente `simulerObservation`. Le framework se charge du reste.

Nous avons choisi de représenter les croyances sous forme d'une liste d'états plutôt que d'une distribution de probabilités car c'est la première idée qui nous est venue. De plus, dans nos scénarios, les états sont discrets et en nombre fini, ce que rend l'énumération exacte directement faisable. Une approche probabiliste aurait été plus complexe à implémenter, et ce n'était pas l'objectif premier de notre projet que d'avoir la meilleure politique possible.

4.1.3 Organisation des fichiers

- `core/` : framework générique (simulateur, agent, historique, politique, belief state)
- `ville/` : scénario du jeu de la ville
- `hanabi/` : scénario Hanabi

Pour lancer les scénarios [1], deux méthodes sont disponibles :
- Les scripts shell `run_ville.sh` et `run_hanabi.sh`.
- Les commandes Maven : `mvn exec:java -Dexec.mainClass="ville.MainVille"` et `mvn exec:java -Dexec.mainClass="hanabi.MainHanabi"`.

4.2 Jeu de la ville

4.2.1 Modélisation

La ville est un graphe non-orienté. Les sommets sont des lieux nommés, les arêtes des chemins praticables. La classe `Ville` gère l'ouverture et la fermeture des lieux.

L'état du jeu (`EtatRDV`) regroupe : la position courante de chaque agent, les messages déposés sur chaque lieu, et les routes fermées connues par chaque agent. Ce dernier champ est important pour le belief state : il permet de modéliser ce que chaque agent sait, et pas seulement l'état réel du monde.

Les observations sont structurées comme une liste de messages, car plusieurs informations peuvent être collectées en un seul tour :
- `MessageAgentPresent` : un autre agent se trouve sur le même lieu,
- `MessageRouteFerme` : une route est fermée (apprise directement ou via un message déposé),
- `MessageRouteHoraire` : une route est fermée selon un horaire,

1. Scripts disponibles à la racine du projet : `run_ville.sh` pour le jeu de la ville, `run_hanabi.sh` pour une partie Hanabi avec une graine aléatoire. Des scripts supplémentaires sont disponibles dans `scripts/hanabi/` (bonscore, pirescore, benchmark) et `scripts/ville/`.

— `MessageAgentBloque` : un agent colocataire a tenté de quitter le lieu sans y parvenir, révélant indirectement qu'une route est fermée.

Nous avons été confrontés à deux choix de conception. Le premier : est-ce qu'on essaie de calculer le meilleur endroit pour déposer un message, ou est-ce qu'on le dépose toujours des qu'une route est fermée ? Durant le développement au premier semestre, nous avions fait en sorte que l'agent pose un message sur chaque case future de son chemin, au cas où un autre agent passerait dessus. Cependant, l'agent perdait beaucoup trop de tours à déposer des messages. C'est pourquoi, avec l'arrivée du second ordre, nous avons ajouté à la politique le fait de calculer l'intersection des chemins des deux agents pour choisir ou déposer un message.

Cela dit, le second ordre n'est pas forcément pertinent pour le jeu de la ville. Dans notre cas, le problème du rendez-vous n'a pas de contrainte de temps : tous les agents arrivent au rendez-vous tôt ou tard, qu'on optimise ou non le dépôt des messages. Ce qui rend le second ordre un peu secondaire dans ce scénario, même s'il améliore l'efficacité.

4.2.2 Politique PCC et raisonnement du premier ordre

La politique `PolitiquePCC` calcule le plus court chemin vers le rendez-vous par un parcours en largeur (BFS) sur la carte connue de l'agent. Si un déplacement échoue, la politique met à jour sa carte et recalcule le chemin.

Le **raisonnement du premier ordre** entre en jeu quand un agent découvre une route fermée : il décide de déposer un message pour en informer les autres. Le lieu de dépôt est calculé intelligemment via `calculerMeilleurDepot` : la politique cherche l'intersection entre le chemin probable d'un autre agent (estimé via le belief state) et son propre chemin, pour placer le message là où il a le plus de chances d'être lu.

4.2.3 Raisonnement du second ordre

Le **raisonnement du second ordre** pousse la réflexion plus loin : avant de déposer un message, l'agent vérifie si l'autre agent a vraiment besoin de cette information. Via le belief state, il consulte les routes fermées connues de chaque état compatible : si dans tous les états possibles, l'autre agent connaît déjà la route fermée, alors le message est inutile et n'est pas déposé.

Dans le scénario mixte, cela se traduit concrètement : Alice sait que Bob va passer par Shared, et elle sait qu'il ne connaît pas encore la fermeture de Carrefour. Elle lui laisse donc un message. En revanche, pour Charlie qui emprunte un chemin totalement indépendant, aucun message n'est nécessaire.

4.3 Hanabi

4.3.1 Modélisation

Chaque carte est représentée par une couleur et un numéro. L'état du jeu regroupe les mains de tous les joueurs, la pioche, les piles de jeu (une par couleur), et le nombre d'erreurs commises.

À chaque tour, seul l'agent actif dispose d'actions non triviales. Les autres reçoivent une **action vide** : ils ne jouent pas, mais observent quand même ce qui se passe. Les actions disponibles pour l'agent actif sont :

- **Poser** une carte sur la pile de la couleur correspondante. Si la carte n'est pas la suivante attendue, c'est une erreur.
- **Défausser** une carte pour récupérer un jeton d'indice.
- **Donner un indice couleur** à un autre agent : lui révéler toutes les positions dans sa main qui correspondent à une certaine couleur.
- **Donner un indice valeur** : lui révéler toutes les positions correspondant à une certaine valeur numérique.

L'observation reçue par chaque agent après chaque tour contient : les mains de tous les autres agents, l'état courant des piles, et le nombre d'erreurs. Chaque agent voit donc tout le jeu... sauf ses propres cartes. C'est cette asymétrie d'information qui rend le jeu difficile et qui justifie l'utilisation de domaines de croyance.

4.3.2 Domaines de croyance

Puisqu'un agent ne connaît pas ses propres cartes, la politique maintient pour chaque emplacement de sa main un **domaine** : l'ensemble des cartes que cet emplacement pourrait contenir, compte tenu des indices reçus et des cartes visibles chez les autres.

Au début, le domaine de chaque emplacement contient toutes les cartes de la pioche initiale, y compris les exemplaires en double ou triple. Chaque indice reçu le réduit : un indice "couleur rouge" sur un emplacement élimine toutes les cartes qui ne sont pas rouges ; un indice valeur "3" élimine toutes les cartes dont le numéro n'est pas 3. Par ailleurs, les cartes visibles dans les mains des autres agents sont retirées du domaine exemplaire par exemplaire : si un autre agent a un rouge-3 en main et qu'il en existe deux copies dans le jeu, une seule est retirée du domaine. L'agent sait donc qu'il ne peut pas avoir *cet* exemplaire précis, mais peut toujours avoir l'autre.

Quand le domaine d'un emplacement ne contient plus que des cartes jouables (c'est-à-dire la prochaine attendue sur la pile de leur couleur), l'agent peut poser cette carte en toute confiance.

4.3.3 Politique et raisonnement de second ordre

La politique `PolitiqueSimpleHanabi` suit un ordre de priorité strict pour choisir son action :

1. Poser une carte si son domaine ne contient que des cartes jouables.
2. Donner un indice sur une carte immédiatement jouable chez un autre agent.
3. Défausser une carte dont le domaine ne contient que des cartes déjà posées (inutiles).
4. Donner un indice de réserve sur une carte utile dans le futur.
5. En dernier recours, effectuer la meilleure défausse possible (en évitant les cartes rares ou uniques).

Lorsque la politique doit donner un indice, elle doit choisir entre un indice couleur et un indice valeur. C'est là qu'intervient le **raisonnement de second ordre** : plutôt que de choisir au hasard, la politique modélise ce que l'agent cible sait déjà sur chacune de ses cartes (en maintenant une estimation de ses domaines de croyance). Elle calcule ensuite lequel des deux types d'indice réduira le plus ces domaines, et préfère celui qui apporte le plus d'information nouvelle.

5 Perspectives

Le framework est conçu pour être étendu facilement. Ajouter un nouveau scénario consiste à implémenter les abstractions de `core` : définir un état, des actions, une fonction de transition, une observation et une politique. Dès que `simulerObservation` est fourni, le belief state devient automatiquement disponible.

Plusieurs pistes d'amélioration restent à explorer :

Raisonnement par pointeurs d'instruction. Dans le jeu de la ville, la politique de chaque agent est un programme déterministe. Il serait possible pour un agent de modéliser non seulement l'état du monde, mais aussi la position de chaque autre agent dans son propre programme (c'est-à-dire ce qu'il va faire au prochain tour). Cela permettrait une coordination encore plus forte sans communication directe.

Politique Hanabi plus agressive. La politique actuelle est prudente. Une piste serait d'intégrer un raisonnement plus profond : par exemple, déduire qu'une carte est jouable non pas parce que son domaine est réduit à des cartes sûres, mais parce que la combinaison de plusieurs indices reçus implique logiquement qu'elle l'est.

Généralisation du belief state. Actuellement, le belief state est utilisé uniquement dans le jeu de la ville. Nous pourrions dans le future l'implémenté à Hanabi ce qui permettrait de maintenir une liste d'états de jeu compatibles, offrant une base plus intelligente à la politique que les simples domaines par carte et en hypothèse améliorer le score moyen global.

Généralisation du belief state. Actuellement, le belief state est utilisé uniquement dans le jeu de la ville. L'appliquer à Hanabi permettrait de maintenir une liste d'états de jeu compatibles, offrant une base plus rigoureuse à la politique que les simples domaines par carte.

6 Conclusion

Nous avons conçu et implémenté un framework multi-agents générique, modulaire et fonctionnel. Le cœur du système (package `core`) gère la boucle de simulation, l'historique, et le mécanisme de belief state par filtrage. Deux scénarios l'illustrent : le jeu de la ville, avec un raisonnement du premier et du second ordre basé sur le belief state, et Hanabi, avec une politique à domaines de croyance et raisonnement de second ordre sur les indices.

6.1 Notions apprises

Ce projet nous a permis de renforcer nos connaissances sur certaines méthodes Java peu courantes comme `computeIfAbsent` ou `removeIf`, que nous avons absolument voulu mettre en place pour que le code soit le plus propre et compacte, sans être illisible.

Mais aussi concevoir un framework générique nous a appris à penser en termes d'architecture globale : séparer ce qui est propre à un scénario de ce qui est commun à tous(généricité), de façon à ce que le coeur du système reste intact quelle que soit l'implémentation. C'est un exercice qu'on n'a pas l'habitude de faire dans les autrse projets.

Enfin, le raisonnement de second ordre a été la notion la plus difficile à comprendre. L'idée qu'un agent ne raisonne pas seulement sur le monde mais sur ce que les autres agents croient du monde est pas simple a concevoir au départ. La mettre en pratique dans le code, en s'assurant que chaque agent modélise correctement les croyances des autres a demandé pas mal d'effort de réflexion.

6.2 Répartition des tâches

Tom David a en partie implémenté le framework générique (core) : abstraction des agents, de l'historique, de la politique et des observations. Il a également développé le jeu de la ville ainsi que le BeliefState et son mécanisme de filtrage. Au second semestre, il a développé la politique PCC avec le raisonnement du premier ordre (calcul du meilleur dépôt de message) et du second ordre (vérification que le message est utile avant de le déposer), ainsi que l'ajout du MessageAgentBloque et des routes fermées connues dans l'état.

Titouan Pasquier a en grande partie implémenté Hanabi : structure du jeu, simulation, politique simple, système d'indices couleur et valeur, seeds. Au second semestre, il a travaillé sur l'amélioration de la politique via les domaines de croyance : prototype, réduction des domaines en fonction des indices reçus, et logique de choix d'action.

Marius Guillais a travaillé sur le système de messages et d'observations dans le jeu de la ville, l'historique et la boucle de jeu, ainsi que la comparaison des observations. Au second semestre, il a contribué à l'amélioration de la politique Hanabi , notamment la prise en compte de la défausse et le choix intelligent de l'indice à donner et du second ordre.

Maxim Chepy a contribué à une première version de la politique PCC, à la filtration et la réinitialisation des domaines de croyance dans Hanabi, ainsi qu'à la rédaction du rapport.